Web Privacy — TP1: Stateful Web Tracking

Malek
Université Mohammed VI Polytechnique (UM6P)
College of Computing

September 2026

Introduction

The purpose of this practical session is to understand how a stateless HTTP server can recognize the same browser across several requests, and how this mechanism is used for web tracking. A small fictional user, Alice, visits websites without logging in. The goal is to build a prototype that recognizes her browser and to observe, at every step, what is stored in the browser and what is transmitted over HTTP.

Two roles were used during the session:

- **Web developer**: creating the websites and the tracking mechanisms.
- **Privacy investigator**: inspecting what the browser and the server exchange, using the browser Developer Tools and the Flask server terminal.

Each challenge followed the cycle *Predict* → *Test* → *Inspect* → *Explain*.

1 Setting Up the Environment

1.1 Working Folder and Virtual Environment

A folder named `statefulTracking` was created to hold the server code and web pages. A Python virtual environment was created with `python -m venv venv` so that the packages needed for the TP do not affect the main Python installation.

Activating the virtual environment first failed because PowerShell blocks script execution by default. This was fixed with:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
venv\Scripts\activate
```

```
PS C:\Users> cd asus
PS C:\Users\asus> cd  .\Documents\
PS C:\Users\asus\Documents> cd  .\study\
PS C:\Users\asus\Documents\study> cd '.\information security\'
PS C:\Users\asus\Documents\study\information security> cd .\TP\
PS C:\Users\asus\Documents\study\information security\TP> cd .\tp2\
PS C:\Users\asus\Documents\study\information security\TP\tp2> cd .\statefulTracking\
PS C:\Users\asus\Documents\study\information security\TP\tp2\statefulTracking> python -m venv venv
PS C:\Users\asus\Documents\study\information security\TP\tp2\statefulTracking> venv\Scripts\activate
venv\Scripts\activate : File C:\Users\asus\Documents\study\information
security\TP\tp2\statefulTracking\venv\Scripts\Activate.ps1 cannot be loaded because running scripts is
disabled on this system. For more information, see about_Execution_Policies at
https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ venv\Scripts\activate
+ ~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
```

Figure 1: Activation of the virtual environment blocked by the PowerShell execution policy.

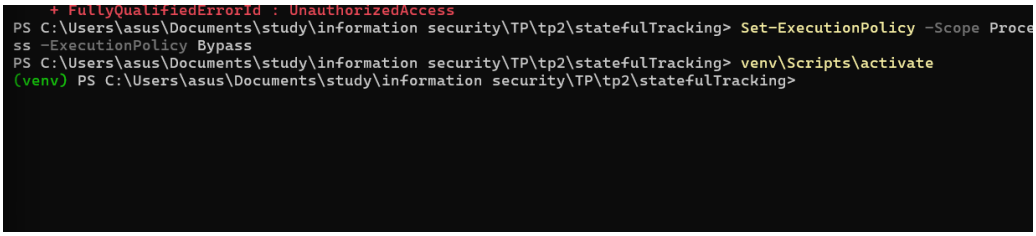


Figure 2: Execution policy changed for the current session; the virtual environment is activated, shown by the `(venv)` prefix.

1.2 Installing Flask

Flask was installed inside the activated virtual environment with `pip install flask`. The version was then checked with `flask -version`.

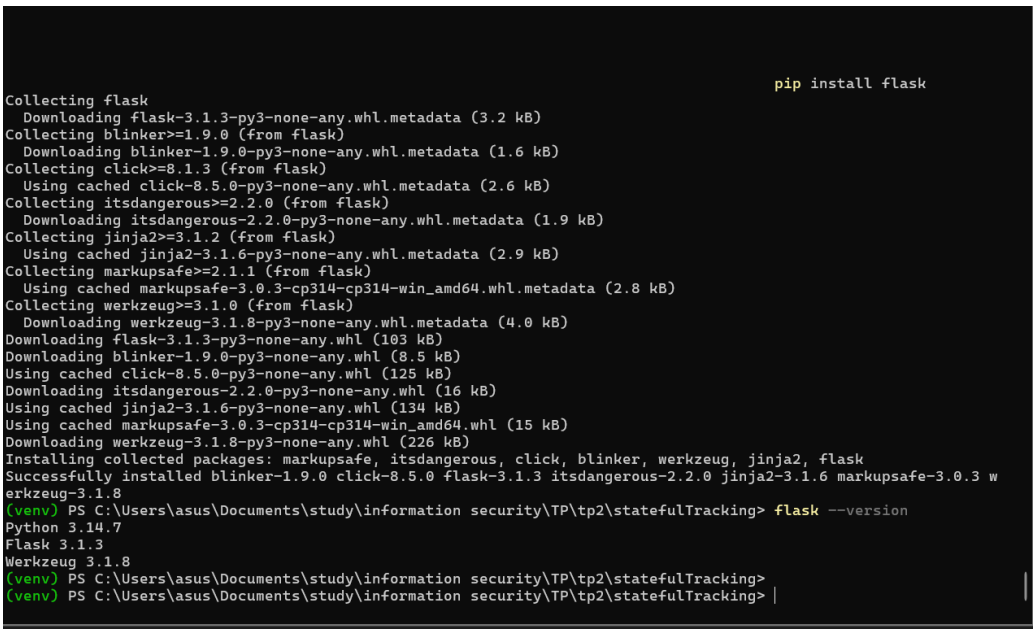


Figure 3: Installation of Flask and its dependencies, and version check (Flask 3.1.3).

2 Running the First Website

2.1 Project Structure

Inside `statefulTracking`, an `app.py` file and a `templates` folder containing `index.html` were created.

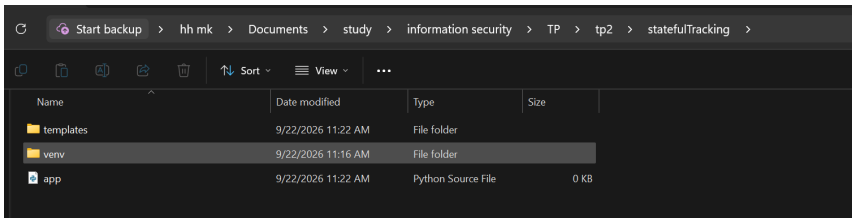


Figure 4: Project folder after creating `app.py` and the `templates` folder, next to `venv`.

2.2 The Web Page and the Server

The static page `index.html` was written:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Alice’s Web Journey</title>
</head>
<body>
<h1>Welcome to Alice’s Web Journey</h1>
</body>
</html>
```

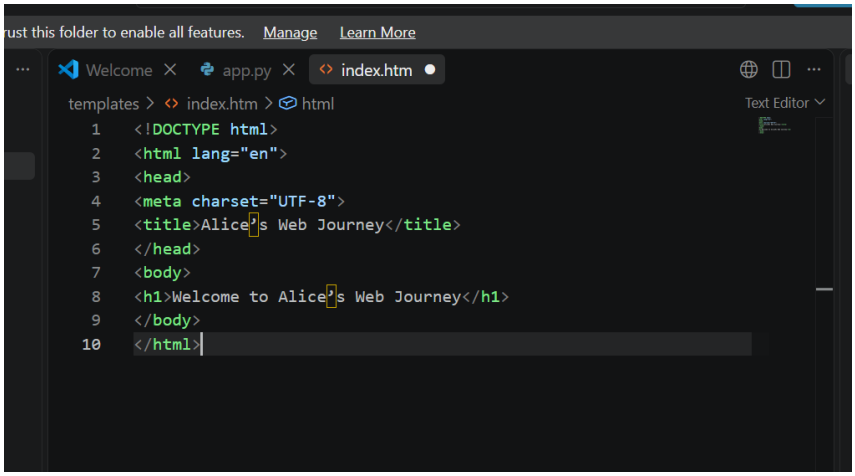


Figure 5: `index.html` being edited in the code editor.

The Flask server was then written in `app.py` to serve this page on route `/`, and started with `python app.py` on `http://127.0.0.1:8000`.

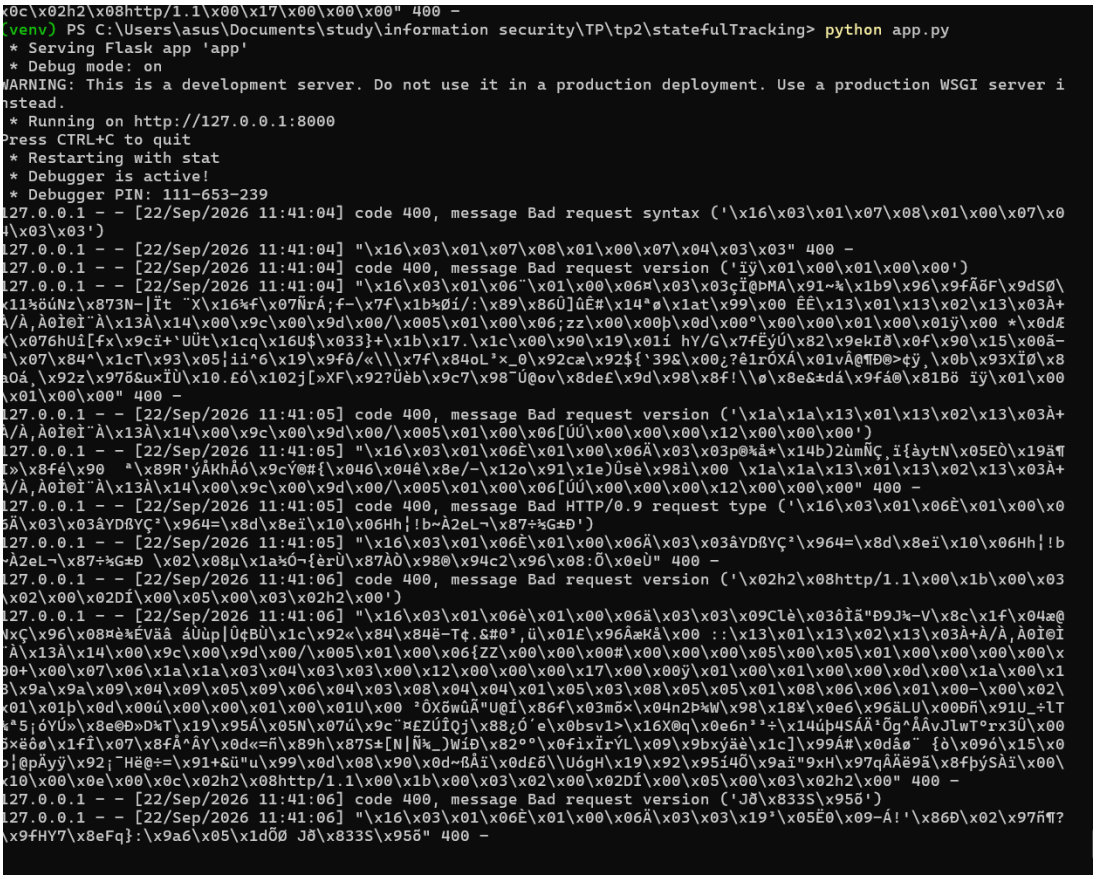


Figure 6: Flask development server running on port 8000. The lines with code 400 come from the browser probing the port over HTTPS while the server only speaks HTTP.

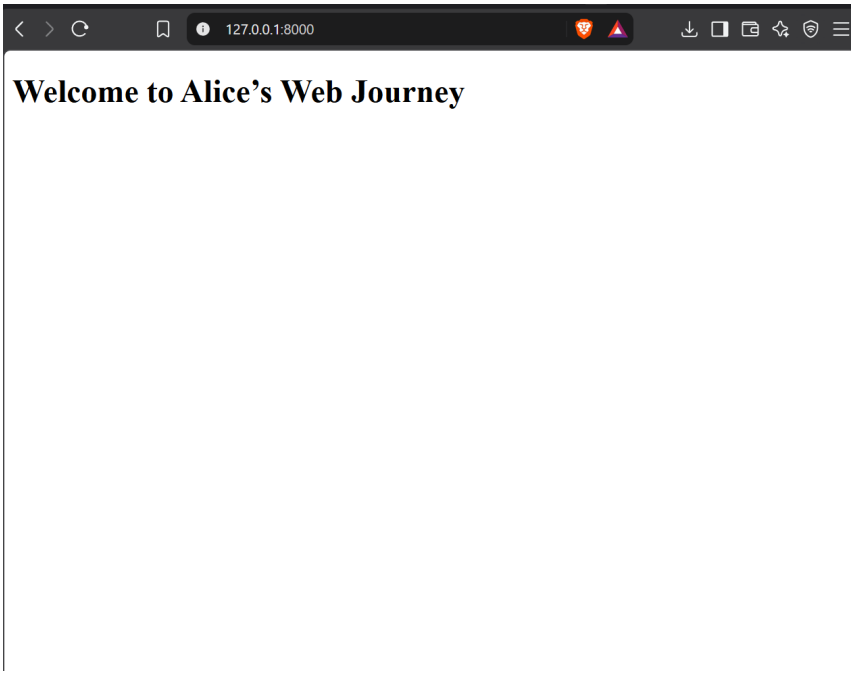


Figure 7: The page loaded at `127.0.0.1:8000`.

2.3 Using a Domain Name

To use a readable name instead of an IP address, the entry `127.0.0.1 lab.test` was added to the Windows hosts file, and the site was reopened at `http://lab.test:8000`.

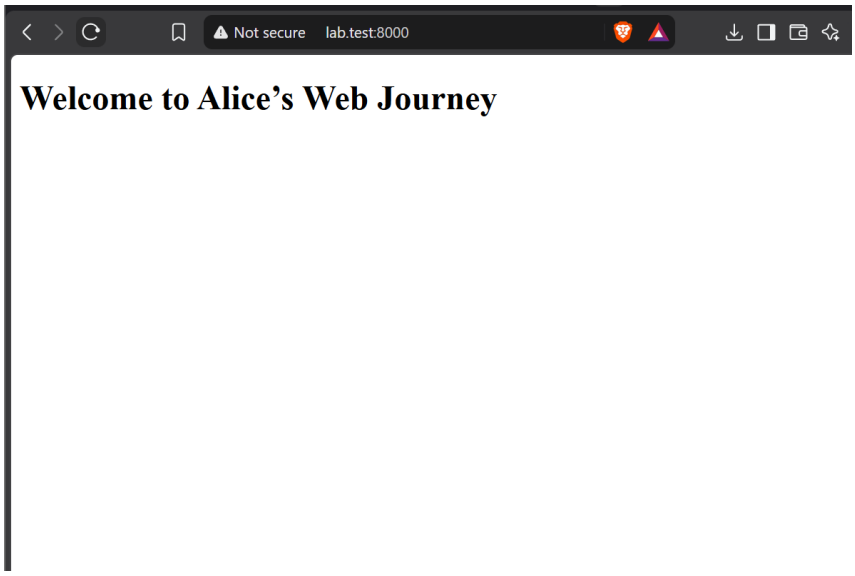


Figure 8: The same page loaded through the domain name `lab.test:8000`, defined in the hosts file.

3 Inspecting HTTP Headers

A custom response header, `X-Lab-Message`, was added in `app.py`, and both the request and response headers were printed in the server terminal for every request.

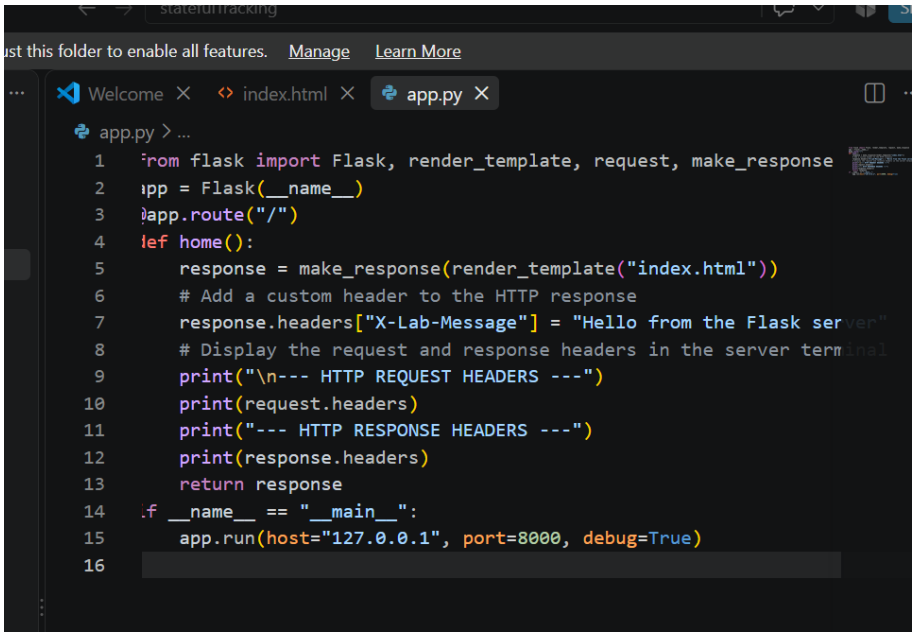


Figure 9: Code printing the request and response headers, and adding the `X-Lab-Message` header.

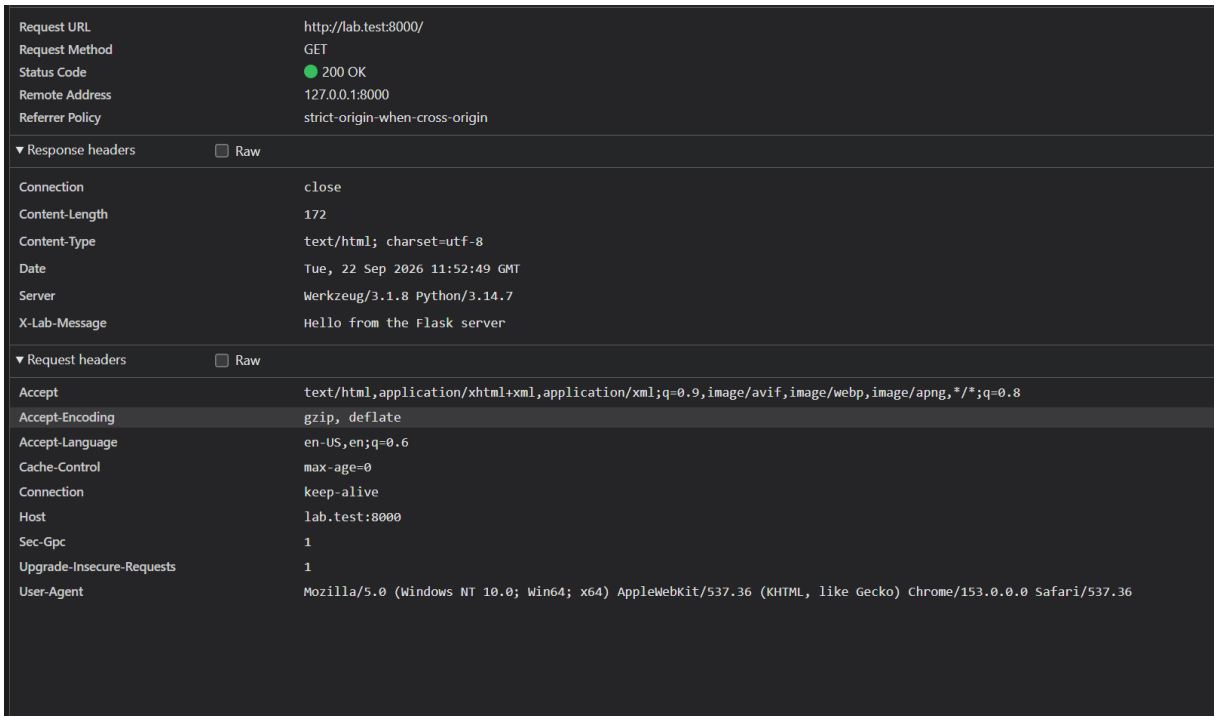


Figure 10: Response headers observed in the browser Network panel, including the custom `X-Lab-Message` header and the standard request headers (User-Agent, Accept, Accept-Language, etc.).



Figure 11: Same request and response headers printed in the Flask server terminal, for several successive requests.

The request headers identify the browser and its preferences (User-Agent, Accept-Language, etc.), while the response headers describe the resource returned by the server. Neither of these, by itself, lets the server recognize the browser again on a later, unrelated request; a cookie is needed for that.

4 Manipulating Cookies

The code was updated so that the server checks for a cookie named `aid`. If the cookie is missing, a new random identifier is generated with `secrets.token_hex(8)` and stored in the browser using `response.set_cookie`. If the cookie already exists, its value is reused.



Figure 12: Server code generating and storing the `aid` cookie only for a new browser.

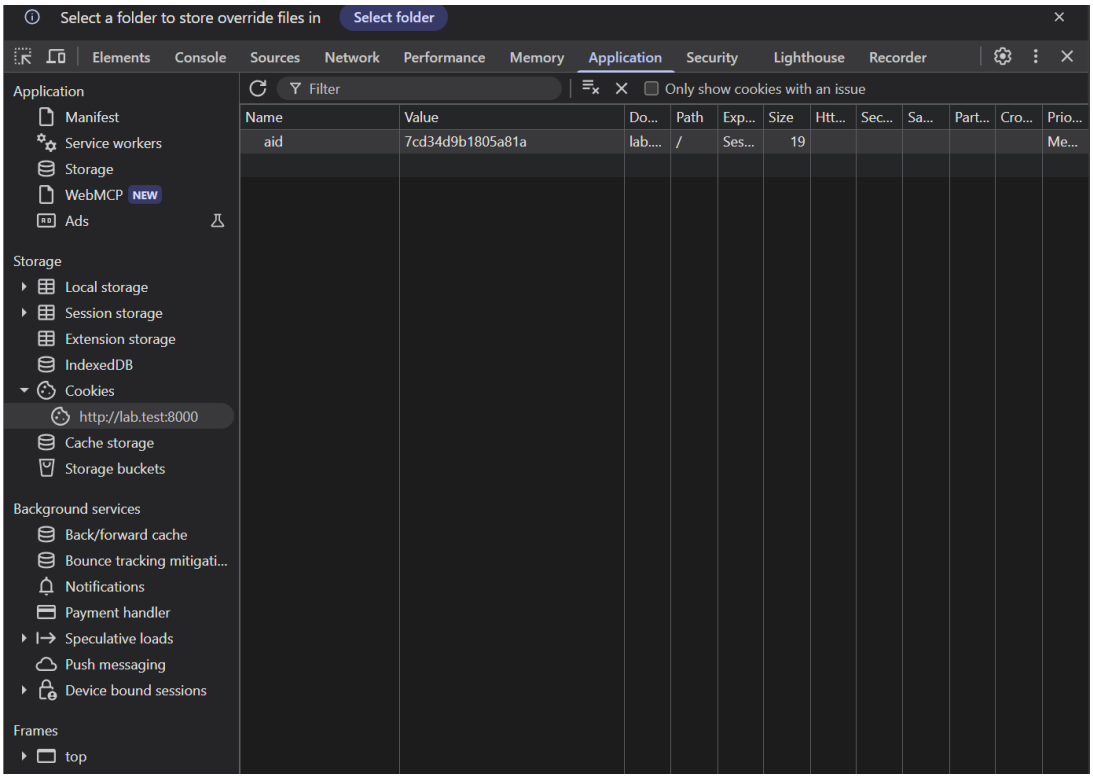


Figure 13: The `aid` cookie stored in the browser (Application panel), with no explicit `Expires` date, so its lifetime is the browser session.

Persistence Test

As instructed, the browser was closed and reopened, then the site was visited again.

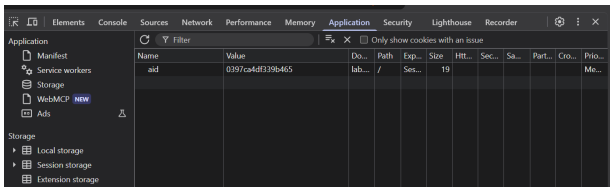


Figure 14: Cookie value after closing and reopening the browser.

Observation. The value of `aid` changed after restarting the browser (from `7cd34d9b1805a81a` to `0397ca4df339b465`). Because the cookie has no `Expires`/`Max-Age` attribute, it is a session cookie: the browser deletes it when it is closed, so on the next visit the server sees no cookie, treats the browser as new, and issues a fresh identifier. A persistent identifier would instead require an explicit expiration date on the cookie.

5 Challenge 1 — Third-Party Tracking

For this challenge, three independent Flask applications were created:

- **publisher-one** (port 8001): a page with technology-style content.
- **publisher-two** (port 8002): a page with travel-style content.
- **tracker-one** (port 5000): a third-party tracker embedded by both publishers through an `iframe`.

Each publisher includes the tracker with a query string identifying the site and the page, for example:

```
<iframe src="http://tracker-one.local:5000/?site=publisher-one&page=home"
        width="600" height="400"></iframe>
```

The tracker assigns a random identifier (`secrets.token_hex(8)`) stored in a cookie named `tracker`, and appends every visit, together with the site and page parameters, to an in-memory log.

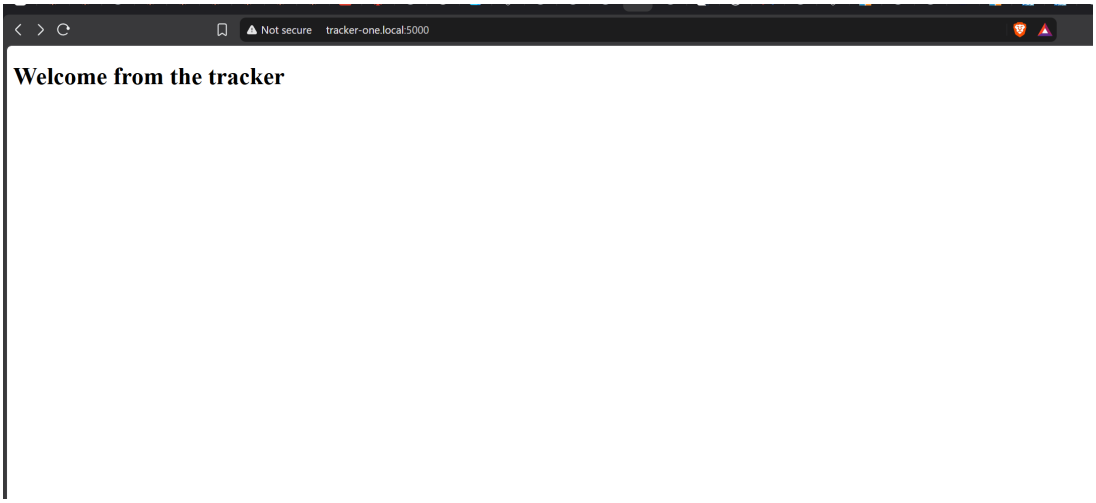


Figure 15: The tracker page loaded directly at `tracker-one.local:5000`.

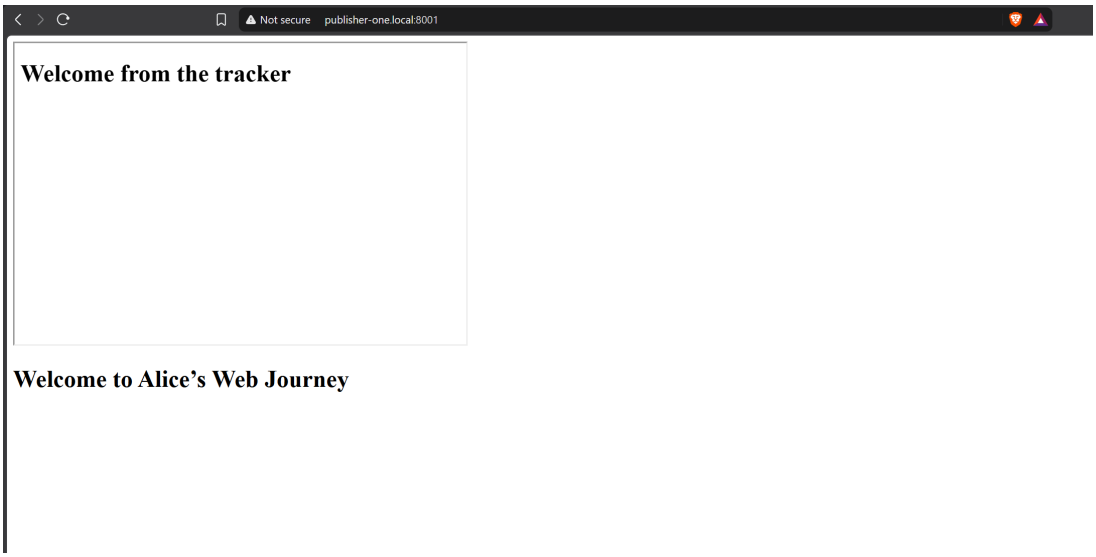


Figure 16: Publisher 1 embedding the tracker in an iframe.

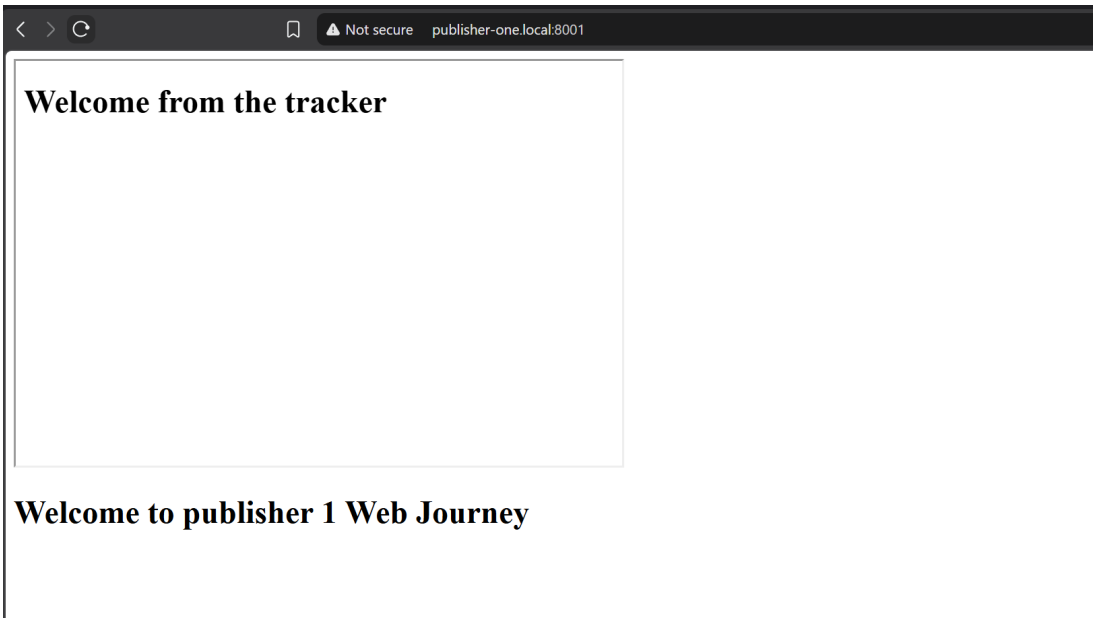


Figure 17: Publisher 1 page after removing the visible border styling of the previous test.

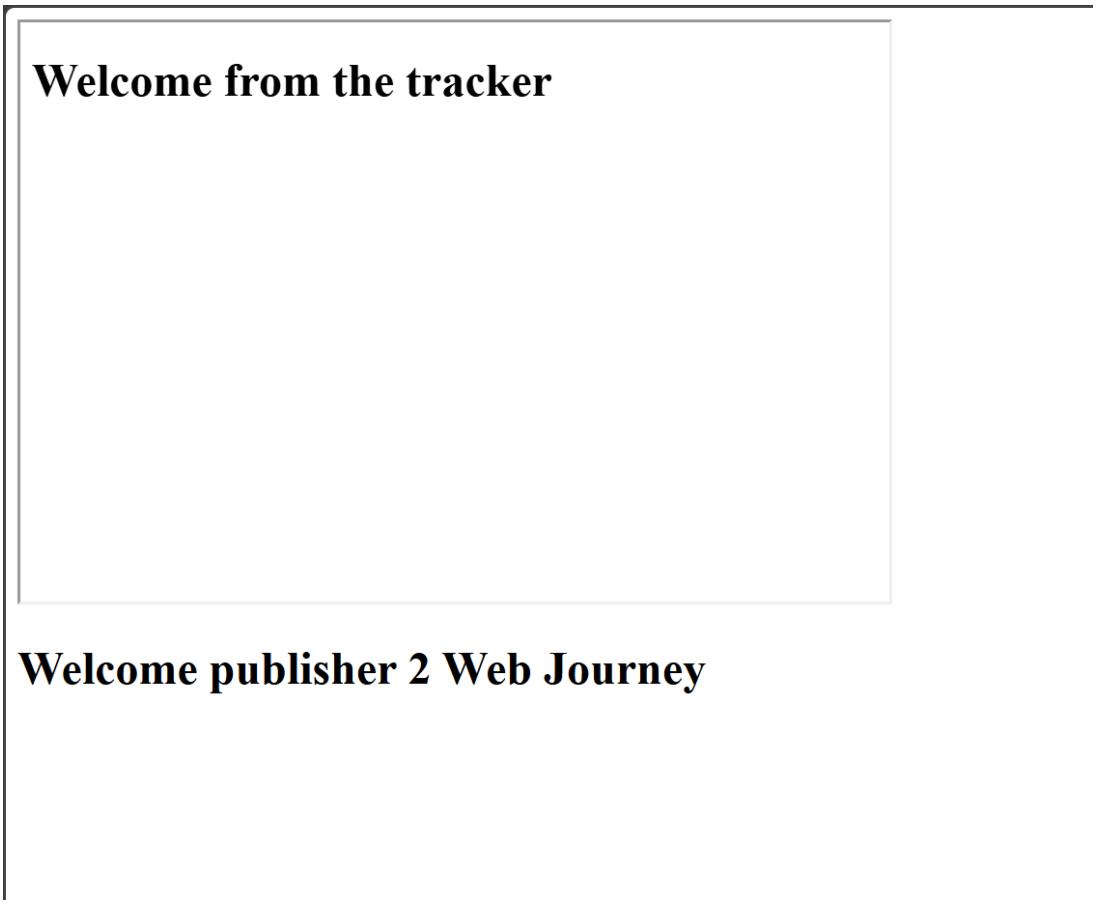
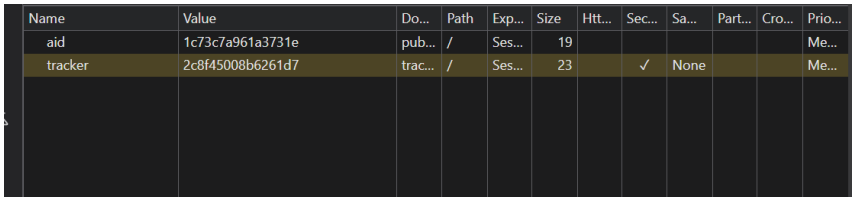


Figure 18: Publisher 2 also embedding the same third-party tracker.



Name	Value	Do...	Path	Exp...	Size	Htt...	Sec...	Sa...	Part...	Cro...	Prio...
aid	1c73c7a961a3731e	pub...	/	Ses...	19						Me...
tracker	2c8f45008b6261d7	trac...	/	Ses...	23		✓	None			Me...

Figure 19: Cookies visible on a publisher page: the first-party `aid` cookie (publisher's own domain) and the third-party `tracker` cookie (tracker's domain, with `Secure` set and `SameSite=None`).

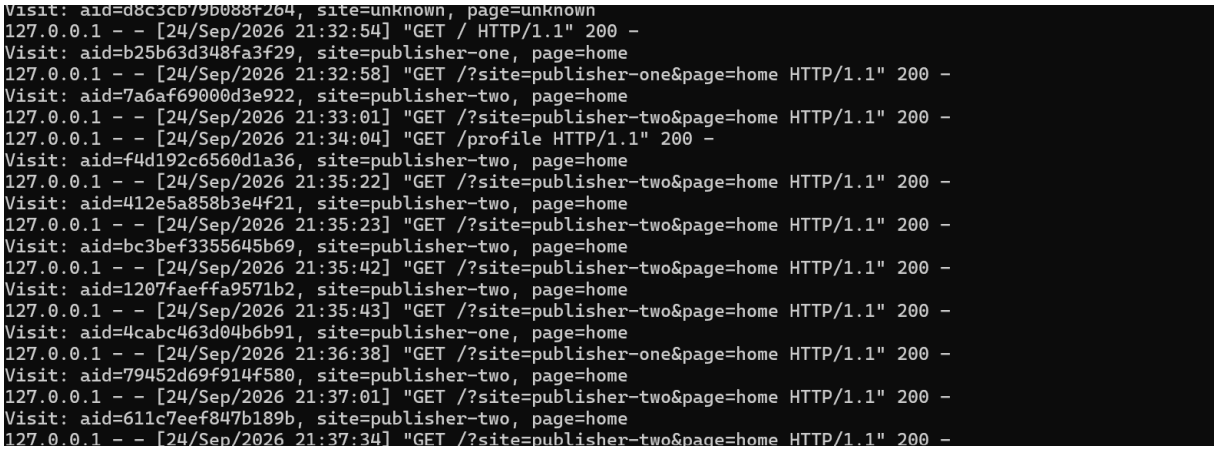


Figure 20: Tracker server log, associating an identifier with the visited site and page for each request.

Observation. The tracker log in Figure 20 shows a different `aid` value for almost every visit, even for repeated visits to the same publisher, instead of one stable identifier per browser. Because the tracker's cookie is set with `Secure=True` while the site is only served over plain HTTP, the browser refuses to store or resend that cookie; the tracker therefore never sees its own cookie come back and issues a brand-new identifier on every request. This is itself a useful observation for the investigation: a misconfigured cookie attribute can silently break persistent tracking, and only serving the tracker over HTTPS (or removing the `Secure` flag for this local test) would let a single identifier persist across visits.

6 Challenge 2 — First-Party Analytics

An analytics service was created as a fourth application (`analytics-one`, port 9100), serving a script `analytics.js` that is included directly in each publisher page:

```
<script src="http://analytics.test:9100/static/analytics.js"></script>
```

Although the file is downloaded from `analytics.test`, it executes in the context of the page that includes it, so any cookie it creates with `document.cookie` belongs to the *publisher's* domain, not to `analytics.test`.



Figure 21: `analytics.js`: it looks for an existing `analytics_id` cookie, creates one if absent, stores it with `document.cookie`, and sends it together with the current hostname to the analytics server using a tracking pixel (`Image`).

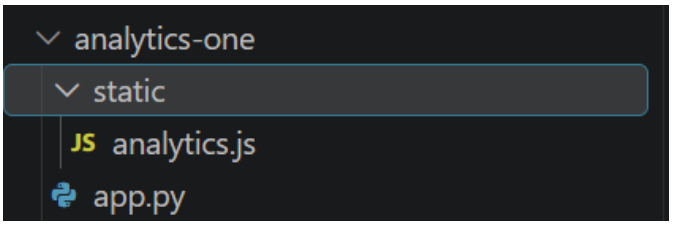


Figure 22: Project structure of the analytics application.

While setting up the route serving the script, the first requests failed:

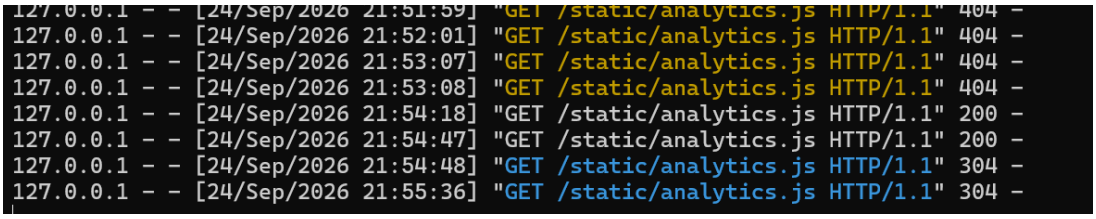


Figure 23: Analytics server terminal: the first requests for `/static/analytics.js` return 404 because the route was not yet defined; after adding it, the requests succeed (200, then 304 from cache).

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>publisher 1 Web Journey</title>
</head>
<body>
<!-- <iframe src="http://tracker-one.local:5000/?site=publisher-one&page=home"
width="600"
height="400">

</iframe> -->
<h1>Welcome to publisher 1 Web Journey</h1>
<script src="http://analytics.test:9100/static/analytics.js"></script>
</body>
</html>
```

Figure 24: Rendered HTML of publisher 1, showing the analytics script tag pointing to `analytics.test:9100`.

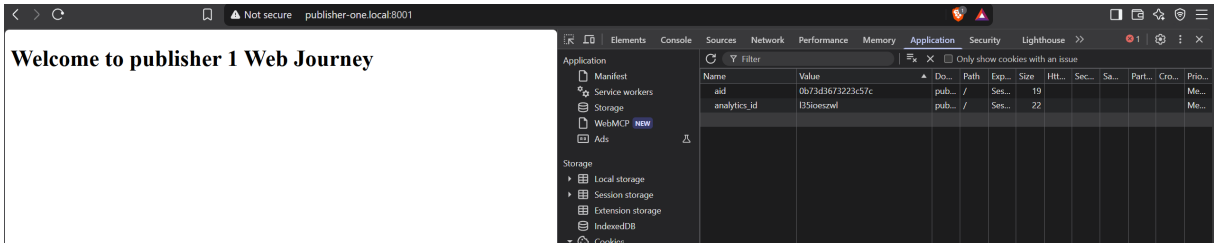


Figure 25: Cookies on publisher-one: a first-party `aid` cookie and the `analytics_id` cookie created by the analytics script, both scoped to the publisher's own domain.

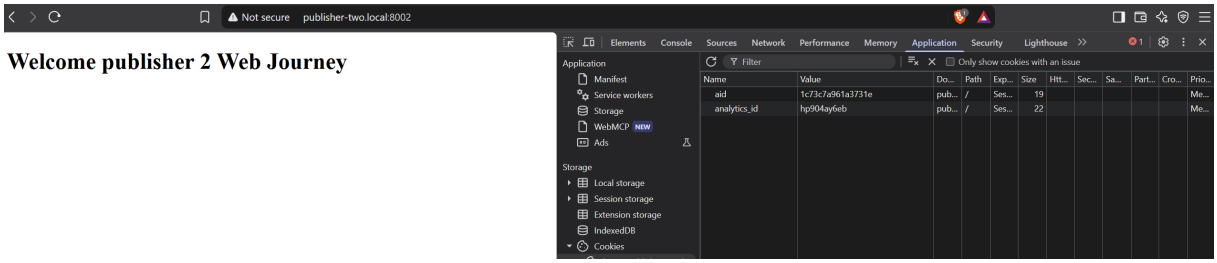


Figure 26: Cookies on publisher-two: a different `aid` and a different `analytics_id` value than on publisher-one.

Observation. The `analytics_id` cookie is *different* on publisher-one and publisher-two. Because `document.cookie` in `analytics.js` always writes a cookie for the domain of the page currently executing the script, the cookie is stored under each publisher's own domain rather than under `analytics.test`. As a result, the analytics server cannot rely on that cookie to link the two visits together: from cookies alone it sees two separate identifiers, one per publisher, and therefore builds two separate partial profiles instead of one combined profile of Alice's browsing. A real tracking pixel would instead need to read/set its identifier from a cookie scoped to its own domain (as in Challenge 1) to merge activity across sites.

7 Conclusion

This session showed several concrete mechanisms by which a stateless HTTP server can recognize a returning browser, and the practical limits of each one:

- A first-party cookie set directly by the visited site (`aid`) is simple but only persists for the browser session unless an expiration is set.
- A third-party cookie set by a resource embedded through an `iframe` (the tracker) can, in principle, follow the same browser across different publisher sites, but is sensitive to cookie attributes such as `Secure` and `SameSite`, which must match the context in which the resource is actually served.
- A script served from a third-party domain but executed in a first-party context (the analytics case) can only set cookies scoped to the page that includes it, which prevents it from building a single cross-site profile from cookies alone.

Overall, the practical work confirmed that the ability to track a browser across visits and across sites depends less on the tracking code itself than on where, in which domain, and under which attributes the identifying cookie is actually stored.